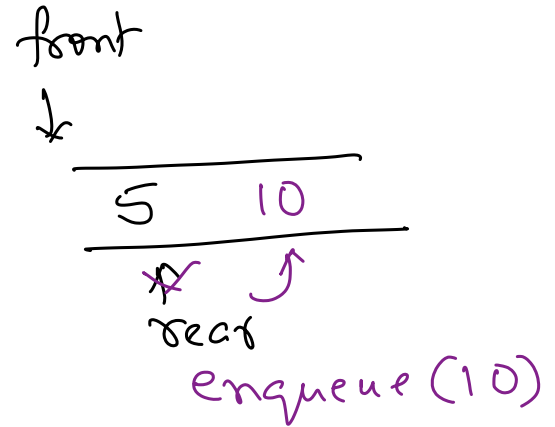
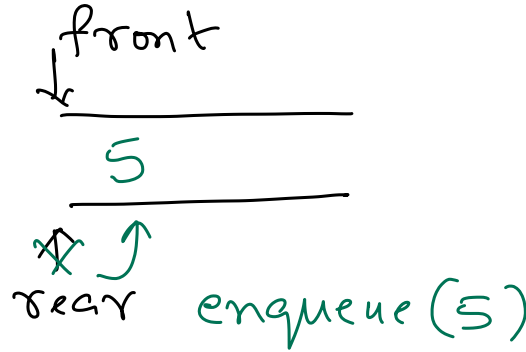
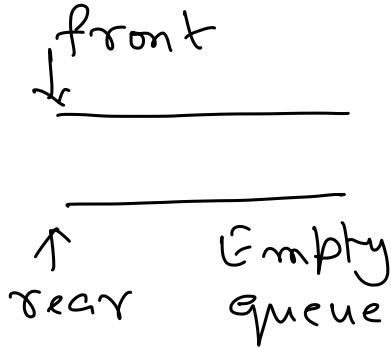


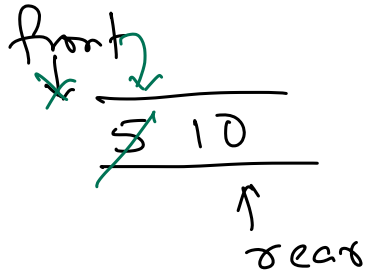
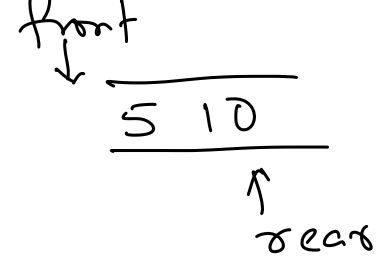
Queue

- Queue is a linear data structure.
- Queue is a container of objects.

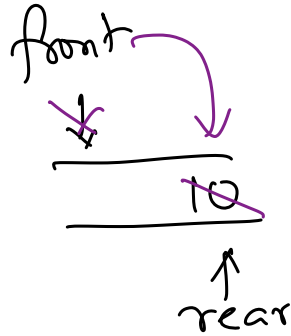
Queue operations

- FIFO – First In First Out
- Elements are added and removed according to FIFO principle.
- Addition of elements are performed at “**rear**” of queue.
- Elements are removed from “**front**” of queue.

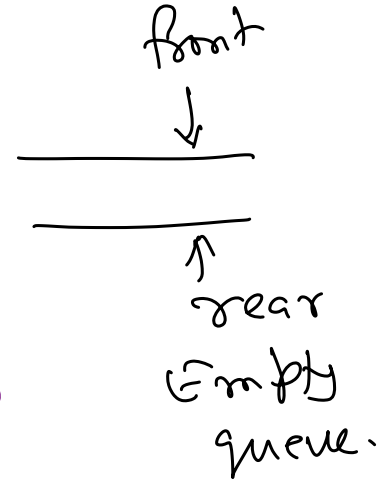




dequeue() $\Rightarrow$ 5



dequeue() $\Rightarrow$ 10

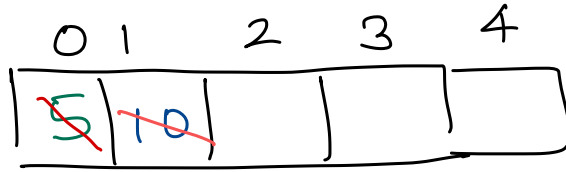


Define queue as ADT

```
public interface Queue {  
    void enqueue (int element);  
    int dequeue ();  
    boolean is Empty();  
    boolean is Full();  
}
```

Queue using array

dequeue ←



← enqueue

front → ~~-1~~ 0 1

rear → ~~-1~~ 0 1

enqueue(5)

enqueue(10)

dequeue() ⇒ 5

dequeue() ⇒ 10

```
public class FixedSizeQueue  
    implements Queue {
```

```
    private int[] queueData;
```

```
    private int front;
```

```
    private int rear;
```

```
    public FixedSizeQueue(int n) {
```

```
        queueData = new int[n];
```

```
        front = -1; rear = -1;
```

Enqueue(element)

- If queue is full then stop. $\rightarrow$ if (is Full()) return; $\rightarrow$ throw appropriate exception.
- Make space at rear for new element. $\rightarrow ++rear;$
- Store new element and make it the rear element. $\rightarrow$ queueData[rear] = element;

Dequeue()

- If queue is empty then stop. $\rightarrow$ if (is Empty()) return -1;
- Move the front towards rear. $\rightarrow ++front;$
- Remove and return the front element as result. $\rightarrow$ return queueData[front];

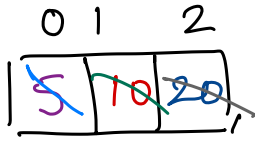
IsEmpty()

- If no elements stored in queue then return true. $\rightarrow$ if (front == rear) return true;
- Else return false. $\rightarrow$ return false;

IsFull()

- If no space left for new element to be stored then return true. $\rightarrow$ if (rear == (queueData.length - 1))
- Else return false. $\rightarrow$ return false;
- $\rightarrow$ return true;

Linear queue suffer from the problem that queue can be full and empty at the same time.



front $\rightarrow$ ~~1~~ ~~0~~ ~~2~~
rear $\rightarrow$ ~~1~~ ~~0~~ ~~2~~

enqueue (5)

enqueue (10)

enqueue (20)

is Empty() $\Rightarrow$ false

is Full() $\Rightarrow$ true

dequeue() $\Rightarrow$ 5

dequeue() $\Rightarrow$ 10

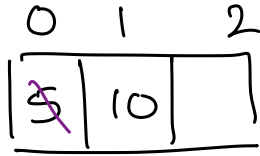
dequeue() $\Rightarrow$ 20

is Empty() $\Rightarrow$ true

is Full() $\Rightarrow$ true

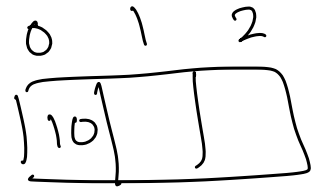
Solution:

- ① In dequeue, after removing the element, shift all remaining elements of queue to left by one place.



front $\rightarrow$ ~~-1~~ 0
rear $\rightarrow$ 1

dequeue() $\Rightarrow$
 $\downarrow$
5



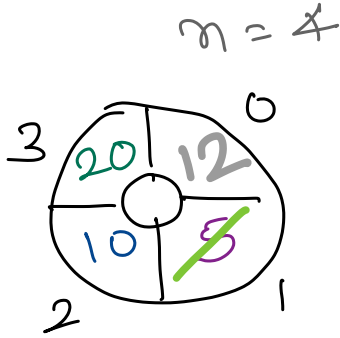
front $\rightarrow$ -1
rear $\rightarrow$ 0

- ② In dequeue, we check if queue is empty & full at same time. If yes, we reset front & rear.

- ③ Circular queue.

Circular Queue

- Last position of Circular Queue is connected back to first position.
Making a circle.



Increment front &
rear so that values
are always in range

$[0, n-1]$

by doing modulo n .

front $\rightarrow$ ~~0~~ 1
rear $\rightarrow$ ~~0~~ ~~1~~ ~~2~~ ~~3~~ 0

enqueue(5)

enqueue(10)

enqueue(20)

~~enqueue(30)~~

↓
Should throw queue
full exception.

$$\text{rear} = (\text{rear} + 1) \% n$$

dequeue() $\Rightarrow$ 5

enqueue(12)

Enqueue(element)

- If queue is full then stop.
- Make space at rear for new element. $\rightarrow \text{rear} = (\text{rear} + 1) \% \text{queueData.length}$
- Store new element and make it the rear element.

Dequeue()

- If queue is empty then stop.
- Move the front towards rear. $\rightarrow \text{front} = (\text{front} + 1) \% \text{queueData.length}$
- Remove the front element as result.
- Return result.

IsEmpty()

- If no elements stored in queue then return true.
- Else return false.

IsFull()

- If no space left for new element to be stored then return true. $\rightarrow \text{if } (\text{front} == (\text{rear} + 1) \% \text{queueData.length})$
- Else return false.
- return true;

Application of queue

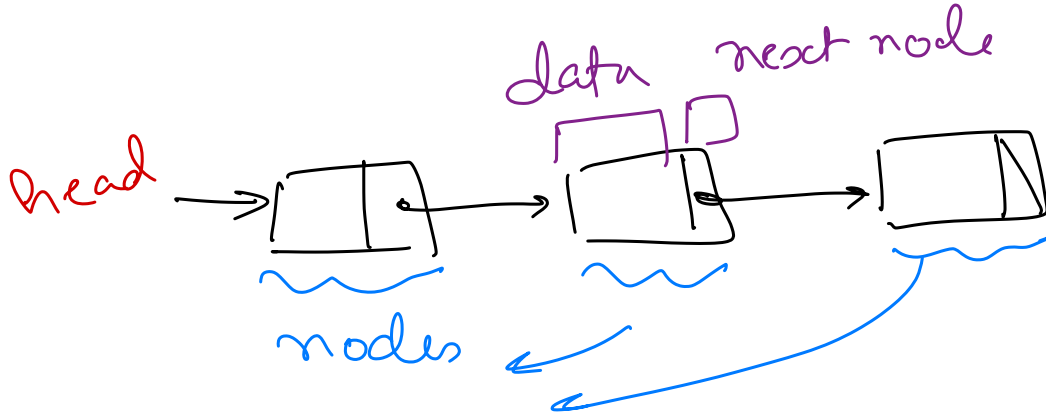
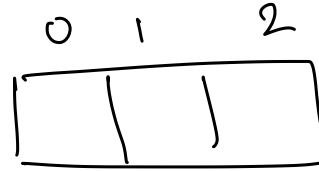
- ① O.S. $\Rightarrow$ Scheduler
 - ② Simulation / Games
 - ③ Other algorithms.
-

- ① Implement queue using stack. (2 stacks)
- ② Implement stack using queue. (2 queue)

Linear Data Structures

Linked List

- Need for a linked list?



Properties of Linked List

- Stores data as a chain of nodes.
- Each node contains data and a pointer to the next node in the chain.
- First node of linked list is pointed by “head”.
When list is empty, head do not point to any node.
- Last node of list points to no node.

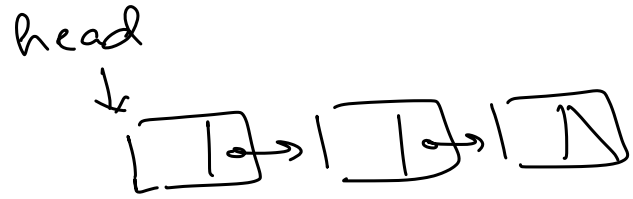
Pros and Cons of Linked List

- Advantages
 - o Can dynamically grow or shrink its size.
 - o Efficient in insertion and deletion of elements.
- Disadvantages
 - o Lookup OR Random access is inefficient.

Types of Linked List

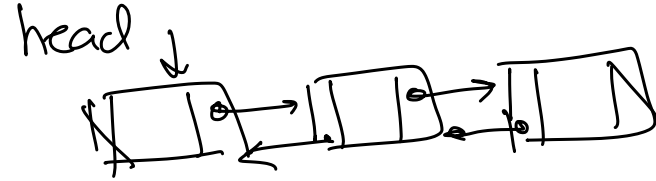
- Single linked list (Uni-directional).

One node keeps track of one neighbour node only.



- Doubly linked list (Bi-directional).

Each node keeps track of two of its neighbours.



- Circular linked list.

Singly Linked List

Traversal

Starting from first element, access each element one at a time, till the last element.

linked list traversal

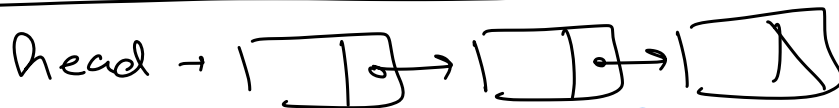
① empty list

Head $\rightarrow$ empty



list is empty

② Non-empty list

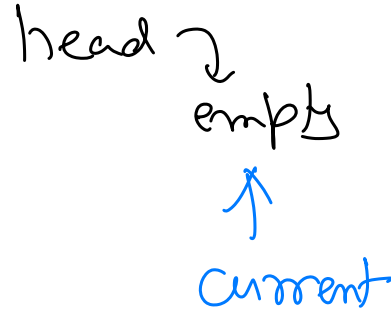


Non-empty list
↓



Singly LinkedList Traversal

- If list is empty then stop.
- Set current to first node of list.
- while (current is not empty) do
 - Process current node.
 - Set current to current node's next.
- Stop.



Singly LinkedList Traversal (Optimised)

- Set current to first node of list.
- while (current is not empty) do
 - Process current node.
 - Set current to current node's next.
- Stop.

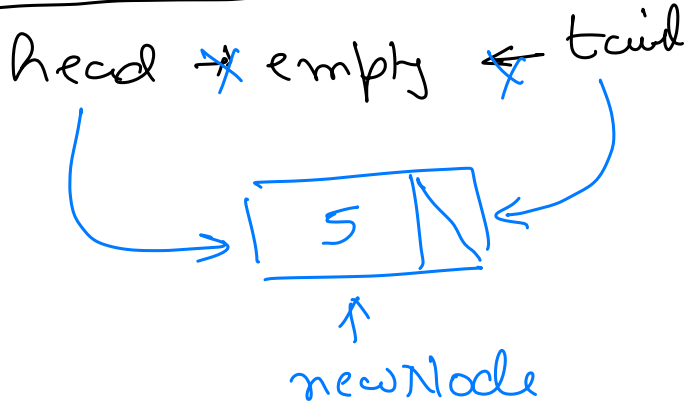
Create Linked List

add At Front

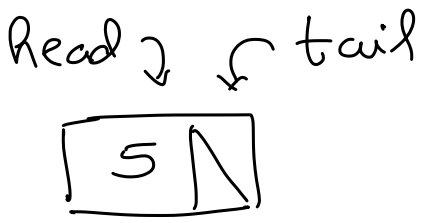
Initially list will be empty $\Rightarrow$

new element will be the only element of the list.

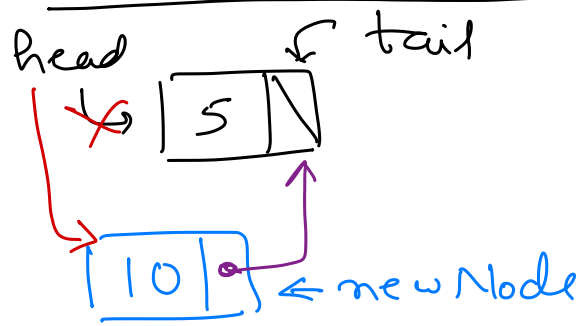
add At Front (5)



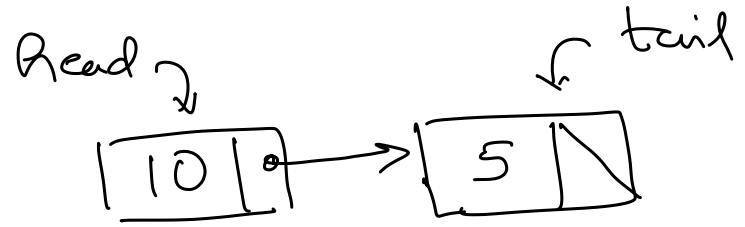
$\Rightarrow$



add At Front (10)

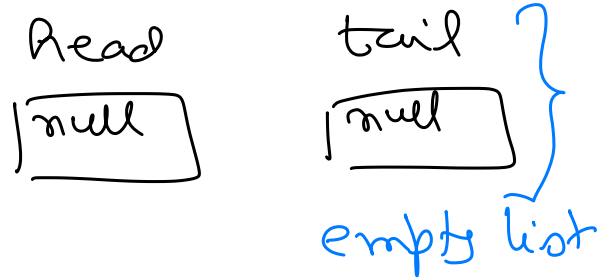


$\Rightarrow$



```
class Node {  
    int data;  
    Node next;  
}  
Node head;  
Node tail;
```

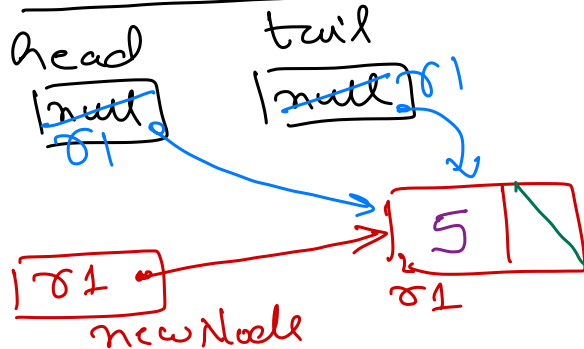
Reference type



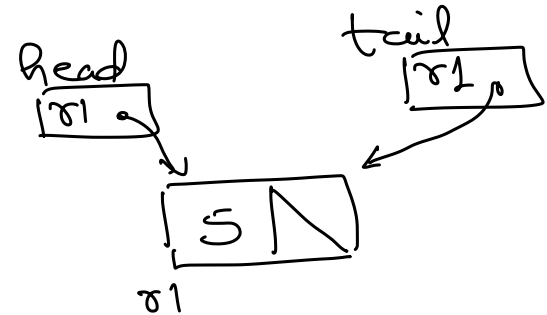
AddAtFront(element)

- Make space for new element, say newNode. $\rightarrow$ Node newNode = new Node();
- Store element in newNode's data. $\rightarrow$ newNode.data = element;
- Set newNode's next to empty. $\rightarrow$ newNode.next = null;
- if list is empty then $\rightarrow$ if (head == null) {
 - Set head and tail to newNode. $\rightarrow$ head = newNode;
 - Stop. $\rightarrow$ tail = newNode;
- Set newNode's next to head. $\rightarrow$ newNode.next = head;
- Set head to newNode. $\rightarrow$ head = newNode;
- Stop. $\rightarrow$ return;

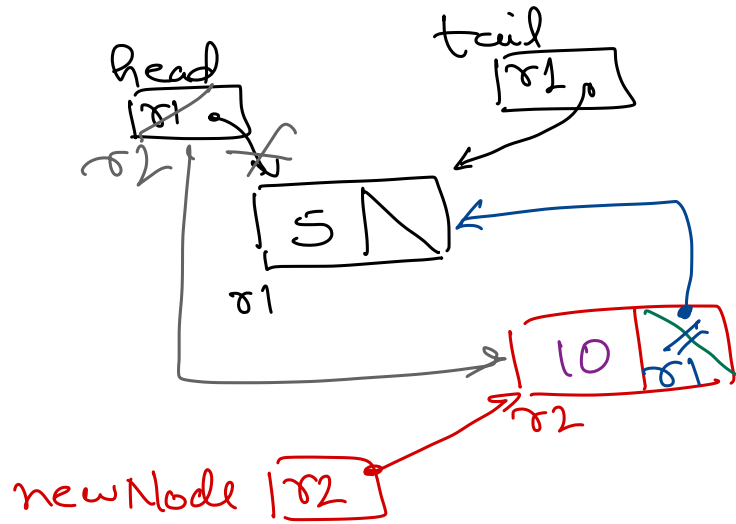
add At Front (5)



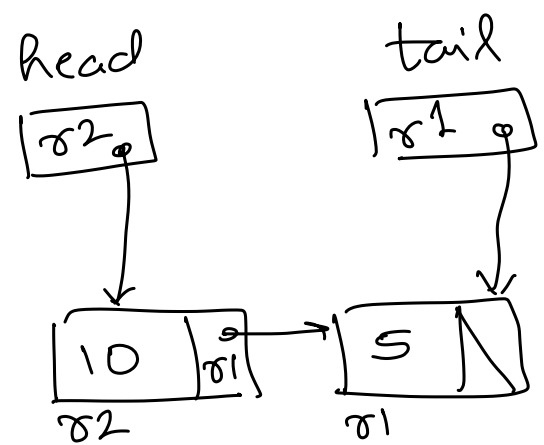
$\Rightarrow$



add At Front (10)



$\Rightarrow$

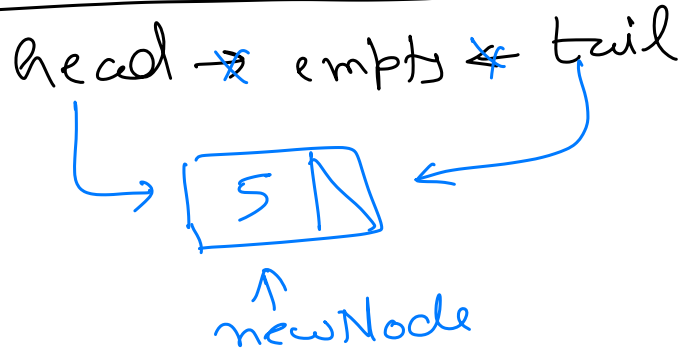


AddAtFront(element) - Optimised

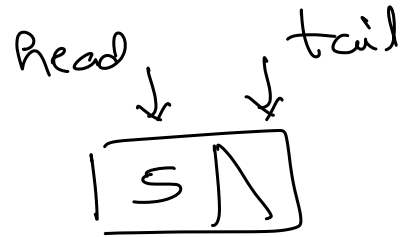
- Make space for new elements, say newNode.
- Store element in newNode's data.
- Set newNode's next to head.
- Set head to newNode.
- if tail is empty then
 - Set tail to head.
- Stop.

add At Rear / addAt End

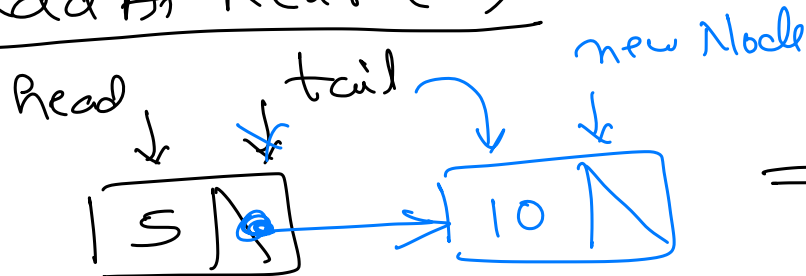
add At Rear (5)



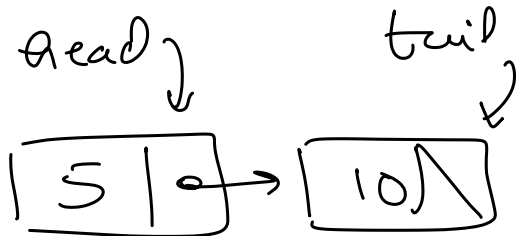
=>



add At Rear (10)



=>



AddAtRear(element)

- Make space for new elements, say newNode. $\rightarrow$ Node newNode = new Node()
- Store element in newNode's data. $\rightarrow$ newNode.data = element;
- Set newNode's next to empty. $\rightarrow$ newNode.next = null;
- if list is empty then
 - Set head and tail to newNode.
 - Stop.
- Set tail's next to newNode.
- Set tail to newNode.
- Stop.

```
if (head == null) {  
    head = newNode;  
    tail = newNode;  
    return;  
}
```

$\rightarrow$ tail.next = newNode;

$\rightarrow$ tail = newNode;

Exercise: Implement addAtRear() without using tail (not keeping track of last node).

Hint: Traverse list to find last node.